

8. Декомпозиция проекта: компоненты и данные в отдельных файлах

Цель занятия

До этого момента мы специально держали почти весь код в `src/App.jsx`, чтобы было проще видеть связь между компонентами, props, state, списками, условиями и формами. Но в настоящем проекте один огромный файл быстро становится неудобным.

Теперь разберем декомпозицию проекта: вынесем компоненты и данные из `App.jsx` в отдельные файлы.

После занятия должны уметь:

- объяснить, зачем разбивать большой файл на маленькие;
- отличать компонент от данных;
- экспортировать компонент из файла;
- импортировать компонент в `App.jsx`;
- понимать разницу между `named export` и `default export`;
- читать структуру React-проекта после декомпозиции.

1. Что говорит документация React

В актуальной документации React есть отдельная тема про импорт и экспорт компонентов. Главная идея: компонент можно объявить в одном файле, экспортировать его, а затем импортировать и использовать в другом файле.

Схема:

text

```
Файл компонента
  ↓ export
Другой файл
  ↓ import
Использование компонента в JSX
```

Пример:

javascript

```
export function Button() {
  return <button>Click</button>
}
```

Использование:

javascript

```
import { Button } from './components/Button.jsx'

function App() {
  return <Button />
}
```

2. Что такое декомпозиция

Декомпозиция - это разбиение большой задачи на маленькие части.

В React это означает:

text

Большой App.jsx

↓

Несколько маленьких файлов

Каждый файл отвечает за одну понятную часть:

- Hero.jsx - верхняя визуальная часть;
- PageTitle.jsx - заголовок и текст;
- ContactForm.jsx - форма;
- ResourceLink.jsx - одна ссылка из списка;
- resourceLinks.js - данные ссылок.

3. Почему большой `App.jsx` становится проблемой

Если в одном файле лежит все сразу, его сложно читать.

В App.jsx могут одновременно находиться:

- массивы данных;
- компоненты кнопок;
- компоненты формы;
- обработчики событий;
- JSX всей страницы;
- условный рендеринг;
- списки.

Такой файл растет и становится похож на длинную простыню.

После декомпозиции App.jsx должен отвечать в основном за сборку страницы:

javascript

```
function App() {  
  return (  
    <>  
      <Hero />  
      <ContactForm />  
      <ResourceLink />  
    </>  
  )  
}
```

4. Новая структура `src`

После декомпозиции структура становится такой:

text

```
src/
├── assets/
│   ├── hero.png
│   ├── react.svg
│   └── vite.svg
├── components/
│   ├── buttons/
│   │   ├── ActionButton.jsx
│   │   └── CounterButton.jsx
│   ├── counter/
│   │   ├── CounterStatus.jsx
│   │   └── StateHint.jsx
│   ├── ContactForm.jsx
│   ├── Hero.jsx
│   ├── PageTitle.jsx
│   └── ResourceLink.jsx
├── data/
│   └── resourceLinks.js
├── App.css
├── App.jsx
├── index.css
└── main.jsx
```

Теперь папка `src` разделена по смыслу.

5. Папка `components`

Папка `components` хранит React-компоненты.

Компонент - это часть интерфейса:

text

```
Hero
PageTitle
ContactForm
ActionButton
CounterButton
ResourceLink
CounterStatus
StateHint
```

Обычно один файл содержит один главный компонент.

Пример:

javascript

```
export function PageTitle({ title, fileName, toolName }) {
  return (
    <div>
      <h1>{title}</h1>
      <p>
        Edit <code>{fileName}</code> and save to test <code>{toolName}</code>
      </p>
    </div>
  )
}
```

6. Папка `data`

Папка `data` хранит данные, которые используются в интерфейсе.

Например, ссылки:

```
javascript
export const socialLinks = [
  {
    id: 'github',
    label: 'GitHub',
    href: 'https://github.com/vitejs/vite',
    iconHref: '/icons.svg#github-icon',
  },
]
```

Это не компонент. Здесь нет JSX. Это обычный JavaScript-массив.

Главная мысль:

```
text
components -> интерфейс
data -> данные для интерфейса
```

7. Named export

В этом проекте мы используем `named export`.

Пример:

```
javascript
export function Hero() {
  return <div className="hero">...</div>
}
```

Импорт:

```
javascript
import { Hero } from './components/Hero.jsx'
```

Фигурные скобки обязательны, потому что это `named export`.

8. Default export

Есть и другой способ - `default export`.

```
javascript
export default function Hero() {
  return <div className="hero">...</div>
}
```

Импорт:

```
javascript
import Hero from './components/Hero.jsx'
```

В нашем проекте договоренность такая:

```
text
App.jsx использует export default App.
Остальные компоненты используют named export.
```

9. Почему важно соблюдать один стиль

Если часть файлов использует `named export`, а часть `default export` без понятной причины, сложнее читать проект.

Плохо для обучения:

```
javascript
import Hero from './components/Hero.jsx'
import { PageTitle } from './components/PageTitle.jsx'
```

Так можно делать в реальных проектах, но для учебной серии лучше держать правило.

В нашем этапе:

```
javascript
import { Hero } from './components/Hero.jsx'
import { PageTitle } from './components/PageTitle.jsx'
```

Так сразу видно повторяемый паттерн.

10. Как изменился `App.jsx`

Раньше `App.jsx` содержал почти все:

```
text
данные
компоненты
форму
разметку страницы
```

После декомпозиции в начале файла появились импорты:

```
javascript
import { ActionButton } from './components/buttons/ActionButton.jsx'
import { CounterButton } from './components/buttons/CounterButton.jsx'
import { ContactForm } from './components/ContactForm.jsx'
import { CounterStatus } from './components/counter/CounterStatus.jsx'
import { StateHint } from './components/counter/StateHint.jsx'
import { Hero } from './components/Hero.jsx'
import { PageTitle } from './components/PageTitle.jsx'
import { ResourceLink } from './components/ResourceLink.jsx'
import { documentationLinks, socialLinks } from './data/resourceLinks.js'
```

Теперь `App.jsx` собирает страницу из готовых деталей.

11. Как читать путь импорта

Пример:

```
javascript
import { Hero } from './components/Hero.jsx'
```

Разберем путь:

```
text
./          -> от текущего файла
components/ -> папка components
Hero.jsx    -> файл Hero.jsx
```

Пример:

```
javascript
import { ActionButton } from './components/buttons/ActionButton.jsx'
```

Здесь файл находится глубже:

```
text
components/buttons/ActionButton.jsx
```

12. Импорт из родительской папки

В файле `src/components/Hero.jsx` изображения лежат не рядом, а в `src/assets`.

Путь:

```
javascript
import heroImg from '../assets/hero.png'
```

`..` означает “подняться на одну папку выше”.

Схема:

```
text
src/components/Hero.jsx
    ↑ ..
src/
    ↓ assets/hero.png
```

13. Данные ссылок в отдельном файле

Файл `src/data/resourceLinks.js` хранит:

```
javascript
export const documentationLinks = [...]
export const socialLinks = [...]
```

В `App.jsx` они импортируются так:

```
javascript
import { documentationLinks, socialLinks } from './data/resourceLinks.js'
```

Преимущество: чтобы изменить ссылку, не нужно искать ее внутри большого JSX.

14. Что осталось в `App.jsx`

После декомпозиции в `App.jsx` остаются:

- состояние `count`;
- состояние `isHintVisible`;
- обработчики событий;
- основная композиция страницы.

Пример:

```
javascript
function App() {
  const [count, setCount] = useState(0)
  const [isHintVisible, setIsHintVisible] = useState(false)

  return (
    <>
      <Hero />
      <ContactForm />
    </>
  )
}
```

Это нормально: `App` остается главным компонентом страницы.

15. Когда выносить компонент

Компонент стоит выносить в отдельный файл, если:

- он стал длинным;
- он имеет отдельный смысл;
- его хочется переиспользовать;
- он мешает читать `App.jsx`;
- в нем есть собственная логика.

Не нужно выносить каждую маленькую строку только ради выноса. Декомпозиция должна помогать читать проект.

16. Частые ошибки

Ошибка 1. Забыть export

Неправильно:

```
javascript
function Hero() {
  return <div>Hero</div>
}
```

Если компонент не экспортирован, другой файл не сможет его импортировать.

Правильно:

```
javascript
export function Hero() {
  return <div>Hero</div>
}
```

Ошибка 2. Перепутать named и default import

Если экспорт такой:

```
javascript
export function Hero() {}
```

импорт должен быть такой:

```
javascript
import { Hero } from './components/Hero.jsx'
```

А не такой:

```
javascript
import Hero from './components/Hero.jsx'
```

Ошибка 3. Неверный путь

Неправильно:

```
javascript
import { Hero } from './Hero.jsx'
```

Если файл лежит в `components`, нужно:

```
javascript
import { Hero } from './components/Hero.jsx'
```

Ошибка 4. Забыть расширение или ошибиться в имени файла

В учебном проекте лучше писать явно:

```
javascript
import { ContactForm } from './components/ContactForm.jsx'
```

Так студенту проще понять, какой файл подключается.

17. Практика

Задание 1

Создайте папку:

```
text
src/components
```

Задание 2

Создайте файл:

```
text
src/components/PageTitle.jsx
```

Перенесите туда компонент PageTitle.

Задание 3

Экспортируйте компонент:

```
javascript
export function PageTitle({ title, fileName, toolName }) {
  return (
    <div>
      <h1>{title}</h1>
      <p>
        Edit <code>{fileName}</code> and save to test <code>{toolName}</code>
      </p>
    </div>
  )
}
```

Задание 4

Импортируйте его в App.jsx:

```
javascript
import { PageTitle } from './components/PageTitle.jsx'
```

Задание 5

Создайте папку:

```
text
src/data
```

Вынесите массивы documentationLinks и socialLinks в файл:

```
text
src/data/resourceLinks.js
```


18. Контрольные вопросы

1. Что такое декомпозиция?
2. Почему большой `App.jsx` становится неудобным?
3. Что хранит папка `components`?
4. Что хранит папка `data`?
5. Что такое `export`?
6. Что такое `import`?
7. Чем `named export` отличается от `default export`?
8. Почему при `named import` нужны фигурные скобки?
9. Что означает `./` в пути импорта?
10. Что означает `../` в пути импорта?
11. Почему данные ссылок лучше вынести из JSX?
12. Что должно остаться в `App.jsx` после декомпозиции?
13. Когда компонент стоит вынести в отдельный файл?
14. Почему не нужно выносить каждую маленькую строку?
15. Как понять, что путь импорта указан неверно?

19. Краткий итог

Декомпозиция помогает превратить большой файл в понятную структуру. Компоненты переезжают в `components`, данные - в `data`, а `App.jsx` становится главным файлом сборки страницы.

Главная схема:

text

```
components -> части интерфейса
data        -> данные для интерфейса
App.jsx     -> собирает страницу
```